



UNIVERSIDAD NACIONAL HERMILO VALDIZÁN

FACULTAD DE CIENCIAS DE LA EDUCACIÓN

Escuela Profesional de Matemática y Física

Asignatura: Ciencias Naturales y del Ambiente



PROYECTO DE INVESTIGACIÓN APLICADA • CYT 2026-II

Sistema Inteligente de Vigilancia Ambiental para las Riberas del Río Huallaga orientado a la detección preventiva del arrojo directo de residuos sólidos

Sector Crítico: Puente Huallaga — UNHEVAL (Huánuco – Amarilis – Pillco Marca)

Área Académica: Ciencias Naturales y del Ambiente •
Matemática y Física.

Línea: Inteligencia Artificial Aplicada a la Preservación
Ecológica Fluvial.

Arquitectura: Edge Computing (Local) + Multimodal
Cloud Reasoning.

Ubicación: Huánuco, Perú

Periodo Académico: 2026-II

Estado: Prototipo Autónomo Operativo (Fases 1 a 4)

SIVARH: Sistema Inteligente de Vigilancia Ambiental para las Riberas del Río Huallaga

Documentación Técnica Oficial y Arquitectura del Sistema (v2)

Orientado a la detección preventiva del arrojo directo de residuos sólidos, sector Puente Huallaga – UNHEVAL

Universidad Nacional Hermilio Valdizán (UNHEVAL) — Huánuco, Perú

Facultad de Ciencias de la Educación — Escuela Profesional de Matemática y Física

Asignatura: Ciencias Naturales y del Ambiente (Semestre 2026-II)

Proyecto de Investigación Aplicada en Ciencia y Tecnología (CyT)

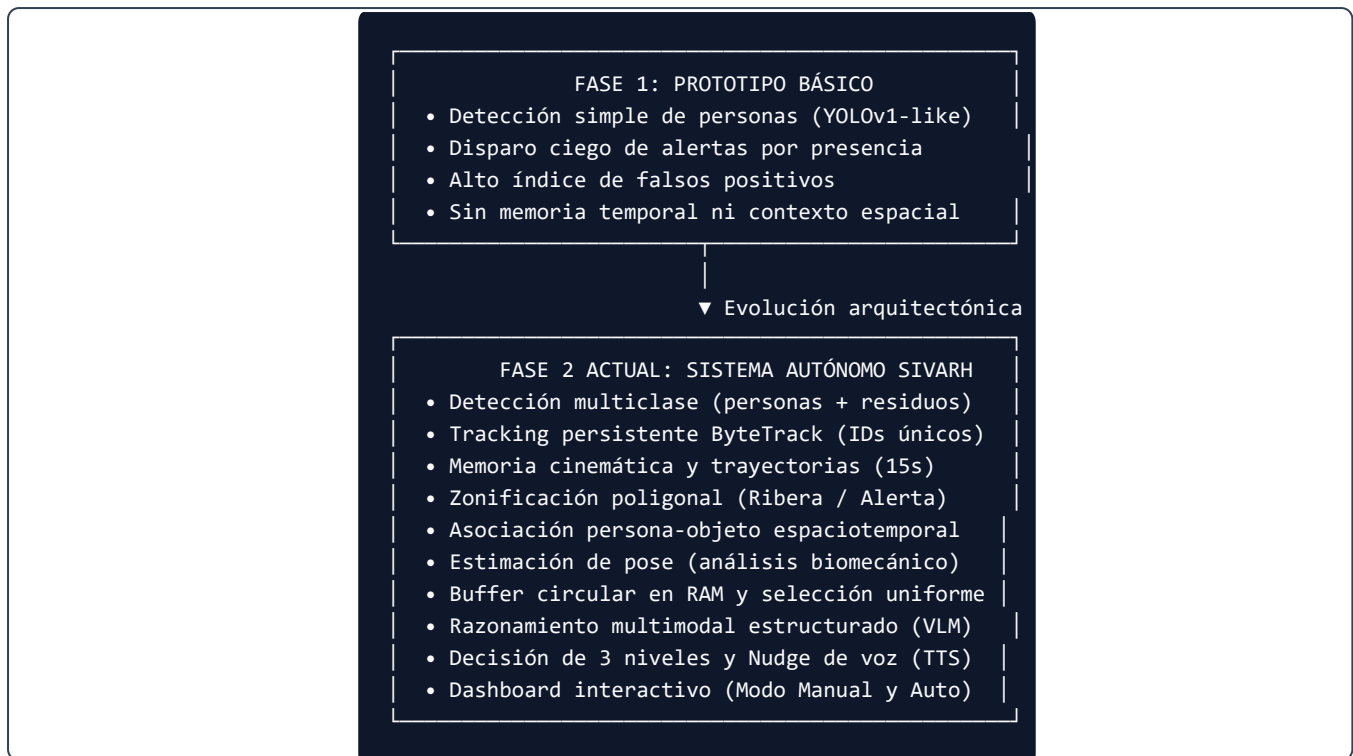
Resumen Ejecutivo

SIVARH es un sistema inteligente de monitoreo y disuasión ambiental diseñado para operar en el borde (*Edge Computing*) en zonas críticas de las riberas del río Huallaga. Su objetivo fundamental es la **intervención preventiva pre-impacto**: detectar conductas de arrojo deliberado o negligente de residuos sólidos (*littering*) y emitir un estímulo auditivo disuasorio (*nudge* cognitivo) en tiempo real para provocar el desistimiento del infractor antes de que el desecho alcance el agua o la faja marginal.

A diferencia de las cámaras de seguridad convencionales o los detectores pasivos, SIVARH integra una arquitectura híbrida de visión artificial local (YOLOv8 + ByteTrack + estimación de pose + zonificación poligonal) y razonamiento semántico multimodal en la nube (OpenAI Vision GPT-5.6 Luna / GPT-4o) con síntesis de voz en streaming ultra-rápido (OpenAI TTS).

1. Evolución del Sistema: De Detector Básico a Sistema Autónomo Espaciotemporal

Para comprender el diseño actual de SIVARH, es clave examinar la transición tecnológica experimentada por el proyecto:



1.1. El Prototipo Inicial (Fase 1: Reactivo Básico)

En su concepción inicial, el sistema operaba bajo un modelo ingenuo y reactivo:

- **Mecanismo:** Cada vez que el detector local identificaba a una persona dentro del campo visual de la cámara, disparaba un evento y generaba una llamada a la API o un mensaje de advertencia.
- **Limitaciones críticas encontradas:**
 1. **Tasa inaceptable de falsos positivos:** El mero tránsito de estudiantes, deportistas o peatones por el Malecón Walker Soberón o el Puente Huallaga detonaba alarmas continuas, aun cuando nadie arrojaba nada.
 2. **Ceguera de objeto:** El sistema no distinguía si la persona portaba un residuo, una mochila de estudio o nada en absoluto.
 3. **Ausencia de memoria temporal:** Cada fotograma se evaluaba de forma aislada, sin capacidad de rastrear trayectorias, persistencia ni separación de objetos.
 4. **Costos y saturación:** Disparar análisis externos por cada persona detectada saturaba el ancho de banda y multiplicaba innecesariamente los costos de cómputo en la nube.

1.2. El Sistema Actual (Fase 2: Autónomo, Espaciotemporal y Multimodal)

Para superar estas fallas estructurales, SIVARH evolucionó hacia una solución autónoma multi-etapa:

1. **Detección Multiclase Local:** Detección simultánea de personas y clases representativas de residuos comunes (`bottle` , `cup` , `backpack` , `handbag`).
2. **Seguimiento Multi-Objeto (ByteTrack):** Asignación de identidades temporales anónimas y estables a través del tiempo, permitiendo conocer si una persona acaba de ingresar, permanece estática o se retira.
3. **Memoria de Trayectoria (`TrackHistory`):** Almacenamiento acotado (15 segundos) de las coordenadas de cada entidad con purga automática por TTL (Time-To-Live).

4. **Zonificación Espacial Semántica** (`ZoneManager`): Segmentación del encuadre en zonas poligonales configurables (ej. *Zona Ribera/Peligro*, *Zona Observación*, *Zona Segura/Vía Pública*). Solo las interacciones que ocurren en o hacia la faja marginal pueden progresar.
5. **Motor de Asociación Persona–Objeto** (`AssociationEngine`): Evaluación espaciotemporal de cercanía e interacción entre personas y objetos sospechosos.
6. **Infraestructura de Pose Biomecánica** (`pose.py`): Detección de puntos clave de brazos y muñecas para reconocer posturas preparatorias de lanzamiento o desprendimiento.
7. **Buffer Circular Muestreado (5 fps / 5s)**: Conservación en memoria RAM de los instantes inmediatamente previos y posteriores al evento (máximo 25 fotogramas), optimizando drásticamente el consumo de RAM.
8. **Selección Inteligente de Cuadros** (`FrameSelector`): Selección cronológica de 4 a 7 cuadros clave representativos de la secuencia de acción (antes, durante y después).
9. **Razonamiento Multimodal Estricto (OpenAI Vision GPT-5.6 / GPT-4o)**: Evaluación de la secuencia temporal completa bajo un contrato Pydantic tipado (`AIAnalysisResult`), distinguiendo explícitamente entre:
 - Portar un objeto (permitido).
 - Manipular un objeto (observación).
 - Soltar, abandonar o arrojar un residuo (infracción confirmada).
10. **Motor de Decisión Gradual** (`DecisionEngine`): Clasificación en tres categorías operativas: `IGNORE` , `LOG_ONLY` y `WARN` .
11. **Disuasión Auditiva Contextual** (`SpeechService` + `AudioOutput`): Emisión instantánea por altavoz de un *nudge* disuasorio generado con voz humana sintética de alta fidelidad.
12. **Doble Modo Operativo en Dashboard**:
 - *Modo Autónomo*: Monitoreo continuo desatendido con cooldown anti-saturación de 20 segundos.
 - *Modo Manual de Campo*: Herramienta controlada para calibración in situ, donde el operador dispara ráfagas de 4 capturas a intervalos fijos de 1.5s y valida la respuesta del modelo VLM.

2. Justificación Científica y Delimitación del Alcance

2.1. Contexto Territorial: Cuenca Media del Río Huallaga

La zona de estudio comprende el corredor fluvial que divide los distritos metropolitanos de **Huánuco, Amarilis y Pillco Marca**, con epicentro en:

- **Puente Huallaga**: Conector vial masivo interdistrital.
- **Malecón Walker Soberón y Malecón Huallaga**: Vías peatonales y vehiculares ribereñas.
- **Acceso a la Ciudad Universitaria UNHEVAL (Cayhuayna)**: Flujo diario de miles de estudiantes y transeúntes.

Los reportes del Organismo de Evaluación y Fiscalización Ambiental (OEFA) y la Fiscalía Especializada en Materia Ambiental (FEMA) identifican más de **216 puntos críticos** de acumulación de basura en la cuenca del Huallaga en Huánuco, con una generación diaria de **100 a 120 toneladas de residuos**.

2.2. Delimitación Estricta de Fuentes de Contaminación

Para mantener la viabilidad técnica y científica, SIVARH define con rigor su frontera de intervención:

Fuente Contaminante	Mecanismo	Alcance en SIVARH
1. Disposición directa por personas (<i>Littering</i>)	Arrojo manual e intencional de botellas PET, bolsas plásticas y empaques en la ribera.	ENFOQUE EXCLUSIVO
2. Basura arrastrada por la corriente	Residuos flotantes transportados desde cuencas altas en época de crecida.	Fuera de alcance (requiere dragado/mallas)
3. Efluentes de aguas residuales	Descargas cloacales domiciliarias e industriales (ej. Camal).	Fuera de alcance (requiere PTAR)
4. Escombros clandestinos con volquetes	Descarga pesada nocturna en fajas marginales.	Fuera de alcance inicial (requiere fiscalización policial)
5. Lixiviados y agroquímicos	Filtraciones de botaderos (Chilepampa) y pesticidas agrícolas.	Fuera de alcance algorítmico

2.3. Fundamento Psico-Conductual: Los *Nudges* Cognitivos

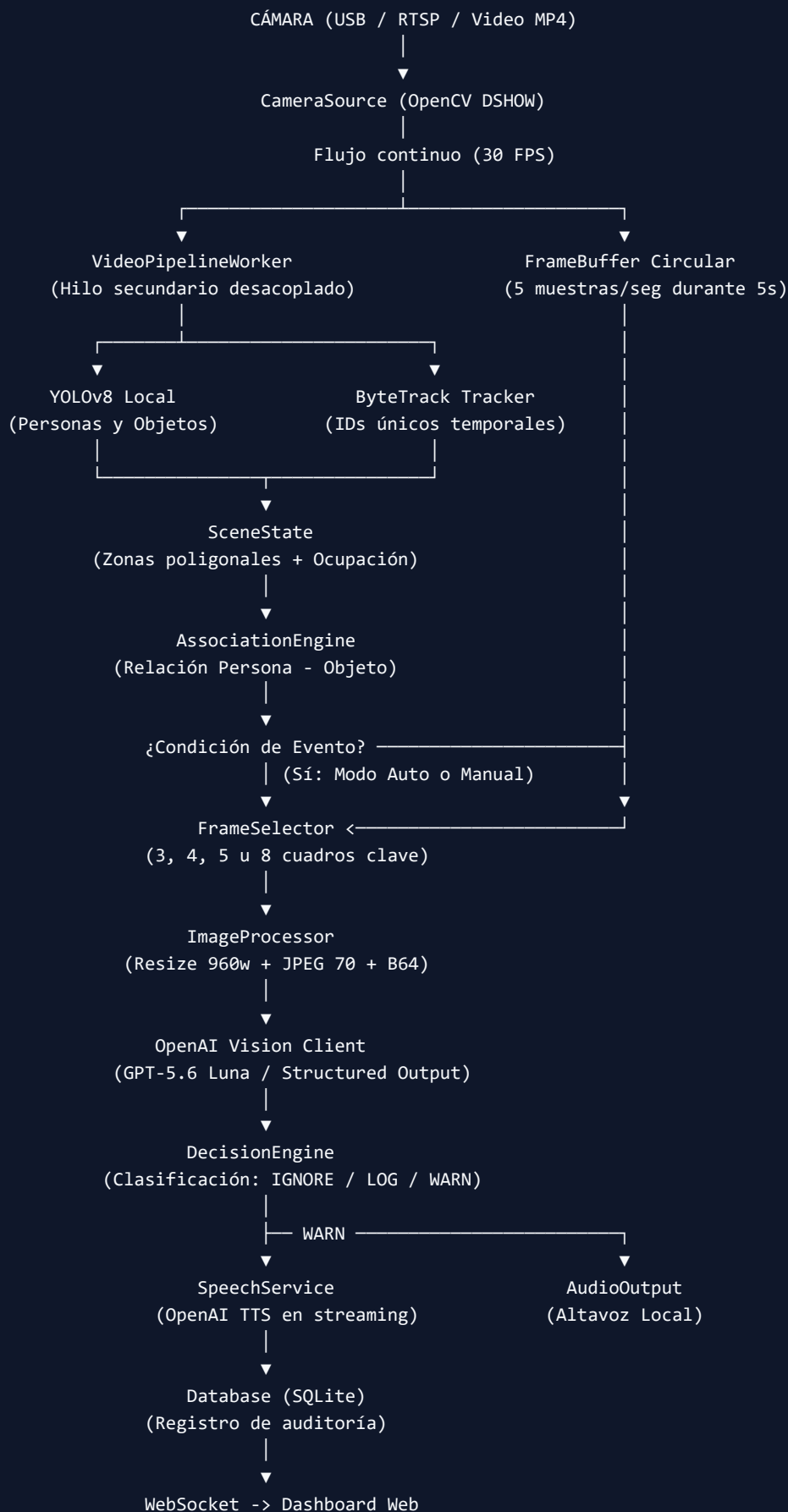
Las campañas de limpieza comunitaria periódicas son valiosas pero de efecto efímero: a los pocos días el punto crítico vuelve a regenerarse por la reincidencia conductual.

SIVARH se fundamenta en la **Teoría del Pensamiento Dual (Daniel Kahneman)**:

- El acto de arrojar un residuo suele ejecutarse mediante el **Sistema 1 (automático, impulsivo, inconsciente)** al aprovechar la dilución del anonimato.
- La advertencia auditiva inmediata y personalizada en el instante previo a soltar el objeto actúa como un **estímulo disuasorio (*nudge*)** que despierta el **Sistema 2 (reflexivo y consciente)**.
- El infractor se percata de que ha sido detectado (*shame effect*), perdiendo la sensación de impunidad y desistiendo de completar la infracción.

3. Arquitectura del Sistema Tecnológico

SIVARH está construido bajo el principio de **separación estricta de responsabilidades** y **conurrencia sin bloqueo**:



3.1. Hilos y Concurrencia

- **Hilo ASGI / FastAPI:** Atiende peticiones HTTP REST y difunde actualizaciones por WebSockets sin demoras.
 - **Hilo de Cámara (`VideoPipelineWorker`):** Captura fotogramas a tasa completa de la cámara, corre inferencias locales rápidas de detección y actualiza el estado global (`SystemState`).
 - **Hilo de Análisis IA Asíncrono:** Cuando se dispara un evento, la preparación de fotogramas, la llamada remota a la API de OpenAI y la síntesis de audio se procesan sin detener la captura de video en vivo ni la interfaz.
-

4. Módulos del Sistema y Responsabilidades

Módulo	Ruta de Código	Responsabilidad Técnica
Servidor Principal	[<code>app/main.py</code>](app/main.py)	Inicialización FastAPI, montaje de estáticos, rutas API y ciclo de vida de la aplicación.
Configuración	[<code>app/config.py</code>](app/config.py)	Centralización tipada mediante Pydantic Settings de variables de entorno (<code>.env</code>).
Estado Global	[<code>app/state.py</code>](app/state.py)	Instancia singleton reactiva <code>system_state</code> con observadores y notificaciones WebSocket.
Abstracción de Video	[<code>app/camera/</code>](app/camera/)	Control de cámaras USB (<code>usb_camera.py</code>), streams IP (<code>rtsp_camera.py</code>) y archivos de prueba (<code>video_file.py</code>).
Worker del Pipeline	[<code>app/camera/worker.py</code>] (app/camera/worker.py)	Bucle principal de adquisición, despacho de frames al buffer, llamada a YOLO y control de modo manual.
Detector Local	[<code>app/vision/detector.py</code>] (app/vision/detector.py)	Inferencia con Ultralytics YOLOv8 para <code>person</code> , <code>bottle</code> , <code>cup</code> , <code>backpack</code> , <code>handbag</code> .
Rastreo (Tracker)	[<code>app/vision/tracker.py</code>] (app/vision/tracker.py)	Adaptador de ByteTrack para asociación frame-a-frame de identidades temporales continuas.
Zonificación	[<code>app/vision/zones.py</code>](app/vision/zones.py)	Gestor de polígonos normalizados, comprobación punto-en-polígono y cálculo de ocupación por zona.
Historial Cinemático	[<code>app/vision/track_history.py</code>] (app/vision/track_history.py)	Registro de posiciones pasadas, cálculo de velocidad y purga automática por TTL.
Asociación	[<code>app/vision/associations.py</code>] (app/vision/associations.py)	Detección de proximidad, solapamiento y correlación espacial entre personas y residuos.
Pose Biomecánica	[<code>app/vision/pose.py</code>](app/vision/pose.py)	Esqueleto corporal para análisis de ángulos de hombros, codos y muñecas en ademanes de tiro.
Buffer de Fotogramas	[<code>app/vision/frame_buffer.py</code>] (app/vision/frame_buffer.py)	Cola doble (<code>collections.deque</code>) muestreada en memoria RAM (5 fps, 5 segundos).
Selector de Cuadros	[<code>app/vision/frame_selector.py</code>] (app/vision/frame_selector.py)	Algoritmo de sub-muestreo temporal uniforme para extraer secuencias compactas.
Compresión	[<code>app/vision/image_processor.py</code>] (app/vision/image_processor.py)	Reescalado a resolución óptima, compresión JPEG al 70% y serialización Base64.
Cliente VLM	[<code>app/ai/vision_client.py</code>] (app/ai/vision_client.py)	Envío de secuencias a OpenAI Vision (GPT-5.6 Luna) y parseo estricto del esquema JSON.
Motor de Decisión	[<code>app/events/decision_engine.py</code>] (app/events/decision_engine.py)	Reglas de negocio para clasificar en <code>IGNORE</code> , <code>LOG_ONLY</code> o <code>WARN</code> según umbrales de confianza.
Síntesis de Voz	[<code>app/speech/tts_service.py</code>] (app/speech/tts_service.py)	Generación de audio mediante OpenAI TTS con soporte de streaming PCM de baja latencia.
Reproducción Local	[<code>app/speech/audio_output.py</code>] (app/speech/audio_output.py)	Emisión física del audio a través de los altavoces de la estación de borde.
Almacenamiento	[<code>app/storage/database.py</code>] (app/storage/database.py)	Base de datos SQLite para auditoría forense de eventos, imágenes clave y decisiones.

Módulo	Ruta de Código	Responsabilidad Técnica
Dashboard Frontend	[<code>frontend/</code>](frontend/)	Interfaz gráfica web moderna con streaming en vivo, métricas, controles y visualizador de secuencias.

5. Modelos de Datos y Contratos Formales (Pydantic)

5.1. Contrato del Análisis Multimodal (`AIAnalysisResult`)

El modelo de visión artificial responde obligatoriamente bajo este esquema validado:

```
class AIAnalysisResult(BaseModel):
    person_detected: bool        # ¿Hay al menos una persona visible en la secuencia?
    suspected_disposal: bool     # ¿Hay un ademán o acción sospechosa de arrojo?
    action_completed: bool       # ¿El residuo quedó efectivamente desprendido/abandonado?
    confidence: float            # Grado de certeza de la inferencia (0.00 a 1.00)
    diagnosis: str               # Diagnóstico técnico en 1 o 2 oraciones
    suggested_warning: str | None # Frase disuasoria recomendada para el infractor
    reasoning: str | None        # Justificación paso a paso de la conclusión visual
```

5.2. Reglas del Motor de Decisión (`DecisionEngine`)

La transición del resultado a una acción física sigue una lógica rigurosa:

- **IGNORE** : Si `suspected_disposal == False` o la confianza no alcanza el umbral mínimo (0.40). No se emite audio ni se alerta.
- **LOG_ONLY** : Si hay sospecha pero `action_completed == False` o la confianza es intermedia ($0.40 \leq \text{conf} < 0.80$). Se guarda registro fotográfico y forense en SQLite para análisis, pero no se emite audio para evitar falsos positivos hacia el transeúnte.
- **WARN** : Si `action_completed == True` y `confidence >= 0.80` (umbral configurable en `.env`). Se activa de inmediato el `SpeechService` para reproducir la advertencia y se marca el evento como alerta activa en el dashboard.

6. Endpoints de la API REST y WebSockets

6.1. Endpoints de Control y Estado

Método	Ruta	Descripción
GET	/api/status	Retorna el estado global del sistema: cámara, tracks, FPS, métricas y logs recientes.
POST	/api/system/start	Inicia la cámara y el worker de procesamiento de video.
POST	/api/system/stop	Detiene la cámara y el worker de video.
GET	/api/events	Lista los eventos históricos almacenados en la base de datos SQLite.
GET	/api/events/{id}	Retorna el detalle completo de un evento específico, incluyendo fotogramas clave.
GET	/api/config	Obtiene la configuración actual del sistema en tiempo de ejecución.
PATCH	/api/config	Modifica dinámicamente parámetros de configuración en caliente.

6.2. Endpoints para Modo Manual y Depuración

Método	Ruta	Descripción
POST	/api/system/manual/start-recognition	Inicia la ráfaga de 4 capturas automáticas espaciadas por 1.5s.
POST	/api/system/manual/send-images	Envía la secuencia capturada a OpenAI Vision para su análisis y decisión.
POST	/api/debug/test-speech	Prueba la síntesis de voz OpenAI TTS y la reproducción por el altavoz.
POST	/api/debug/test-vision	Prueba el cliente multimodal con una imagen estática de prueba.

6.3. Comunicación en Tiempo Real

- **WebSocket** `/ws/status` : Envía pulsos de telemetría continuos (JSON) al frontend con: FPS real, número de personas y objetos detectados, ocupación por zona, tracks activos, estado de la IA y logs del sistema.
- **Streaming de Video** `/video_feed` : Transmisión MJPEG en vivo con bounding boxes, zonas y overlays de depuración opcionales.

7. Configuración del Sistema (.env)

Los parámetros principales que gobiernan el comportamiento operativo de SIVARH se configuran en el archivo `.env` :

```

# Configuración del Modelo de Visión OpenAI
OPENAI_API_KEY=sk-...
OPENAI_VISION_MODEL=gpt-5.6-luna
OPENAI_VISION_REASONING_EFFORT=none
IMAGE_DETAIL=low

# Configuración de Síntesis de Voz (TTS)
OPENAI_TTS_MODEL=gpt-4o-mini-tts
OPENAI_TTS_VOICE=onyx
OPENAI_TTS_SPEED=1.1
OPENAI_TTS_RESPONSE_FORMAT=pcm
OPENAI_TTS_STREAM_BUFFER_MS=400

# Parámetros de Cámara y Video
CAMERA_SOURCE=0
CAMERA_FPS=30
CAMERA_WIDTH=1280
CAMERA_HEIGHT=720

# Detector Local YOLOv8
YOLO_MODEL=yolov8n.pt
YOLO_PERSON_CONFIDENCE=0.50

# Buffer Temporal y Eventos
BUFFER_SECONDS=6
EVENT_CAPTURE_SECONDS=5.6
SEQUENCE_FRAME_INTERVAL_SECONDS=0.8
EVENT_COOLDOWN_SECONDS=20

# Modo de Reconocimiento Manual (Pruebas de Campo)
MANUAL_RECOGNITION_MODE=True
MANUAL_CAPTURE_FRAMES=4
MANUAL_CAPTURE_INTERVAL_SECONDS=1.5
IMAGE_MAX_WIDTH=960
JPEG_QUALITY=70

# Umbral del Motor de Decisión
AI_WARNING_THRESHOLD=0.80

# Servidor de Red
APP_ENV=development
DEBUG=True
HOST=127.0.0.1
PORT=8000

```

8. Procedimiento de Ejecución y Pruebas de Campo

8.1. Puesta en Marcha Local

Para iniciar el servidor y el dashboard:

```

# Activar el entorno e iniciar el sistema
py -3.11 run.py

```

El servidor quedará disponible en `http://127.0.0.1:8000`.

8.2. Flujo de Prueba en Modo Manual Controlado

1. Acceder al dashboard en el navegador web.
2. Comprobar que el video en vivo muestra la cámara activa y que YOLO dibuja las cajas y tracks correspondientes.
3. Simular una acción frente a la cámara (ej. sostener una botella y colocarla en el suelo).
4. Hacer clic en **"Iniciar reconocimiento"**: el sistema capturará automáticamente 4 fotogramas en un lapso de 6 segundos.
5. Hacer clic en **"Enviar imágenes a IA"**: el sistema enviará la secuencia a OpenAI Vision, procesará el veredicto en segundo plano, mostrará el diagnóstico y emitirá la advertencia sonora si la decisión es `WARN`.

9. Calidad, Testing y Confiabilidad

El sistema cuenta con una exhaustiva suite de pruebas automatizadas con **pytest**, cubriendo todos los subsistemas críticos:

- `test_tracker.py` y `test_track_history.py` : Asociación frame-a-frame de identidades de ByteTrack y memoria cinemática.
- `test_zones.py` y `test_scene_state.py` : Cálculo geométrico de polígonos y ocupación de zonas.
- `test_associations.py` y `test_worker_associations.py` : Correlación espacio-temporal persona-objeto.
- `test_pose.py` : Estimación de keypoints y posturas biomecánicas.
- `test_frame_buffer.py` y `test_frame_selector.py` : Ingesta muestreada y algoritmos de selección uniforme.
- `test_ai_schema.py` : Validación del contrato estructurado JSON de OpenAI.
- `test_audio_output.py` : Pipeline de reproducción de audio.
- `test_runtime_config.py` : Modificación en caliente de parámetros vía API.

Resultado de la suite de pruebas: 49 pruebas ejecutadas y aprobadas (100% PASS).

10. Hoja de Ruta y Próximos Pasos (Fases 5–8)

1. Optimización de Latencia para Despliegue en Faja Marginal:

- Implementar síntesis de voz local con **Piper TTS** para reducir la latencia de audio a menos de 300 ms en caso de redes móviles lentas.
- Evaluación de modelos de visión locales compactos (ej. Qwen2-VL o SmolVLM) como respaldo offline.

2. Calibración Ambiental en Terreno:

- Ajuste de umbrales para condiciones de baja visibilidad (crepúsculo/noche) y filtrado de movimiento de vegetación y reflejos en el caudal del río Huallaga.

3. Seguridad y Despliegue en Hardware de Borde:

- Incorporación de autenticación por tokens en la API REST y WebSocket para despliegues en dispositivos embebidos (NVIDIA Jetson / Raspberry Pi 5 con acelerador NPU).